

0	LIMIT_BAL	5000	non-null	int64	
1	GENDER	5000	non-null	int64	
2	EDUCATION	5000	non-null	int64	
3	MARRIAGE	5000	non-null	int64	
4	AGE	5000	non-null	int64	
5	PAY_0	5000	non-null	int64	
6	PAY_2	5000	non-null	int64	
7	PAY_3	5000	non-null	int64	
8	PAY_4	5000	non-null	int64	
9	PAY_5	5000	non-null	int64	
10	PAY_6	5000	non-null	int64	
11	BILL_AMT1	5000	non-null	int64	
12	BILL_AMT2	5000	non-null	int64	
13	BILL_AMT3	5000	non-null	int64	
14	BILL_AMT4	5000	non-null	int64	
15	BILL_AMT5	5000	non-null	int64	
16	BILL_AMT6	5000	non-null	int64	
17	PAY_AMT1	5000	non-null	int64	
18	PAY_AMT2	5000	non-null	int64	
19	PAY_AMT3	5000	non-null	int64	
20	PAY_AMT4	5000	non-null	int64	
21	PAY_AMT5	5000	non-null	int64	
22	PAY_AMT6	5000	non-null	int64	23 default
	payment next month	5000	non-null	int64	

dtypes: int64(24)

memory usage: 976.6 KB

data.isnull()

```
{"type": "dataframe"}
```

```
y=data['default payment next month']
```

```
x=data[['BILL_AMT1','BILL_AMT2']]
```

```
print(x.dtypes) print(x.head())
```

```
BILL_AMT1    int64
```

```
BILL_AMT2    int64
```

```
dtype: object
```

	BILL_AMT1	BILL_AMT2
2308	8864	10062
22404	136736	125651
23397	70122	69080
25058	20678	18956
2664	94228	47635

```

x_train,x_test,y_train,y_test=train_test_split(
x,y,test_size=0.3,random_state=42
)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

models = {
    "Linear SVM": SVC(kernel='linear',C=1,class_weight='balanced'),
    "Polynomial SVM":
SVC(kernel='poly',degree=2,C=1,gamma='scale',class_weight='balanced'),
    "RBF SVM": SVC(kernel='rbf', C=5,
gamma=0.1,class_weight='balanced')
}

for name, model in models.items():
model.fit(x_train, y_train)
y_pred = model.predict(x_test)
print("\n",name)
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Precision", precision_score(y_test, y_pred,
zero_division=0))
    print("Recall", recall_score(y_test, y_pred))
print("F1 Score", f1_score(y_test, y_pred))
print("Accuracy", accuracy_score(y_test, y_pred))

```

```

Linear SVM
Confusion Matrix:
[[ 103 1056]
 [  34  307]]
Precision 0.2252384446074835
Recall 0.9002932551319648
F1 Score 0.36032863849765256
Accuracy 0.2733333333333333

```

```

Polynomial SVM
Confusion Matrix:
[[  35 1124]
 [   8  333]]
Precision 0.22855181880576528
Recall 0.9765395894428153
F1 Score 0.3704115684093437
Accuracy 0.24533333333333332

```

```

RBF SVM
Confusion Matrix:

```

[[560 599]]

```

[166 175]]
Precision 0.22609819121447028
Recall 0.5131964809384164
F1 Score 0.31390134529147984
Accuracy 0.49

def plot_boundary(model,title):
h=0.02
    x_min, x_max = x_train[:, 0].min() - 1, x_train[:, 0].max() + 1
    y_min, y_max = x_train[:, 1].min() - 1, x_train[:, 1].max() + 1

    xx,yy=np.meshgrid(np.arange(x_min,x_max,h),np.arange(y_min,y_max,h))
    z=model.predict(np.c_[xx.ravel(),yy.ravel()])    z=z.reshape(xx.shape)

    plt.contourf(xx,yy,z,alpha=0.3)
    plt.scatter(x_train[y_train==0,0],
x_train[y_train==0,1],
label='No Default (0)',
marker='o')
    plt.scatter(x_train[y_train==1,0],
x_train[y_train==1,1],
label='Default (1)',
marker='x')    plt.legend()
    plt.title(title)    plt.xlabel("BILL
AMOUNT")    plt.ylabel("PAYMENT
AMOUNT") plt.figure(figsize=(12,4))

for i, (name, model) in enumerate(models.items()):
plt.subplot(1,3,i+1)
model.fit(x_train,y_train)
plot_boundary(model,name)
plt.tight_layout()
plt.show()

```

